ARCHWAY v3

Structured Input & Cross-Validation Architecture

Addendum to v2 — Node Specifications, Validation Rules & Task Engine

Field	Detail
Version	3.0 (Structured Input Overhaul)
Based On	v2.0 (all v2 content remains valid: USP, pain points, user stories, diagrams, sprint plan)
What Changed	Free text removed. All input structured. Auto-generated diagrams. Cross-node validation. DMBOK-inspired data quality at input.
Philosophy	"Data quality must be enforced at the point of input, not cleaned after" — DMBOK
Standards Referenced	IEEE 830, ISO/IEC/IEEE 29148, UML 2.5, MoSCoW (DSDM), Alistair Cockburn Use Case Template, Given-When-Then (BDD), Peter Chen/Crow's Foot ERD, DeMarco/Gane-Sarson DFD

Daftar Isi

1. Architectural Changes Overview
2. Node Specifications (All 10 Nodes)
3. Cross-Validation Rules (18 Rules)
4. Task Engine Extraction Rules
5. Updated ERD (with TASKS entity)
6. Validation UX Design
7. Revised Sprint Plan

1. Architectural Changes Overview

Archway v3 introduces a fundamental shift from passive documentation container to active documentation system.

Aspect	v2 (Before)	v3 (After)
Input Mode	Free text + optional guided fields	Structured fields only (no free text)
Diagram Creation	User writes Mermaid syntax manually	Auto-generated from structured data
Validation	None — user can mark Done freely	Per-node validation + cross-node validation
Data Quality	Garbage in, garbage out	DMBOK: quality enforced at input
Task Generation	Based on parsing free text (unreliable)	Based on structured data (deterministic)
Gap Detection	None — nodes are independent	18 cross-validation rules detect gaps
Summary	Manual compilation	Auto-calculated with coverage percentages

▶ **Key Insight:** Structured input means every piece of data has a known shape. This makes auto-generation, cross-validation, and task extraction deterministic rather than heuristic.

2. Node Specifications (All 10 Nodes)

Each node specification includes: structured fields, validation rules, cross-validation links, and task engine extraction rules.

Node 1: Project Brief

Based on: IEEE 830 (Purpose, Scope), ISO 29148, Simon Sinek's Why, industry best practices

Field	Type	Required?	Cross-validation
Project Name	text (max 100)	Yes	—
Background / Why	textarea (max 300)	Yes	—
Objectives	list input	Yes (min 1)	—
Target Users	tag/chip input	Yes (min 1)	→ Feeds User Stories role + Use Case actor dropdowns
Scope In	list input	Yes (min 1)	→ Every item should have ≥1 FR in Requirements
Scope Out	list input	Yes (min 1)	—
Success Metrics	metric + target pairs	Yes (min 1)	—
Constraints	list input	Optional	—

Tech Stack	tag/chip input	Optional	—
References	name + URL pairs	Optional	—

Node 2: Requirements

Based on: IEEE 830 SRS, MoSCoW (DSDM), ISO 29148

Functional Requirements (tab 1)

Field	Type	Required?	Cross-validation
ID	auto (FR-001)	Auto	Referenced by User Stories
Category	dropdown + custom	Yes	→ Used as Task Board grouping
Description	text (max 200)	Yes	—
Priority	MoSCoW dropdown	Yes	→ Inherited by User Stories + Tasks
Related Scope Item	dropdown from Brief	Optional	→ Validates Brief.scopeIn coverage

Non-Functional Requirements (tab 2)

Field	Type	Required?
ID	auto (NFR-001)	Auto
Category	dropdown: Performance, Security, Usability, Reliability, Scalability, Compatibility	Yes
Description	text (max 200)	Yes
Metric	text	Yes
Target	text	Yes
Priority	MoSCoW dropdown	Yes

Validation: Min 3 FRs, min 1 NFR, at least 1 Must priority

Node 3: User Stories

Based on: Mike Cohn User Stories, BDD Given-When-Then

Field	Type	Required?	Cross-validation
ID	auto (US-001)	Auto	→ Referenced by Use Cases, Task Board
Role	dropdown from Brief.targetUsers	Yes	→ Must exist in Brief
Want	text (max 200)	Yes	—
Benefit	text (max 200)	Yes	—
Related Requirement	dropdown from FR list	Yes	→ Validates FR coverage
Priority	auto from linked FR	Auto	—
Acceptance Criteria	Given/When/Then list	Yes (min 1)	→ Generates test tasks

Task Engine: Each US → 1 impl task. Each acceptance criterion → 1 test task.

Node 4: Use Case

Based on: UML 2.5, Alistair Cockburn Use Case Template

Field	Type	Required?
ID	auto (UC-001)	Auto
Name	text, verb+object (max 100)	Yes
Primary Actor	dropdown from Brief.targetUsers	Yes
Secondary Actors	multi-select	Optional
Description	text (max 200)	Yes
Preconditions	list input	Yes (min 1)
Postconditions (Success)	list input	Yes (min 1)
Main Flow	ordered steps (Actor + Action)	Yes (min 3)
Alternative Flows	named branches from main flow steps	Optional
Related User Stories	multi-select from US	Yes (min 1)
Include/Extend relationships	type + target UC	Optional

Diagram: Auto-generated Mermaid graph from actors, use cases, and relationships. User does NOT write Mermaid.

Node 5: Flowchart

Based on: ISO 5807, structured flowchart principles (sequence, decision, loop)

Contains multiple flows, each linked to a Use Case.

Field	Type	Required?
Flow Name	text	Yes
Related Use Case	dropdown from UC	Yes
Trigger	text	Yes
Steps (ordered)	type (start/process/decision/end) + label	Yes (min 4)
Decision: Yes path	target step ID	If decision
Decision: No path	target step ID	If decision

Diagram: Auto-generated Mermaid flowchart TD from steps. Start=oval, process=rect, decision=diamond.

Task Engine: Each process step → 1 impl task. Each decision → 1 validation task.

Node 6: DFD (Data Flow Diagram)

Based on: Tom DeMarco, Gane-Sarson notation, DFD rules

Component	Fields	Rules
External Entity	ID, Name	Cannot connect directly to another EE or DS
Process	ID, Name, Description, Related UC	Must have ≥1 input AND ≥1 output flow
Data Store	ID, Name, Related ERD Entity	Must connect to ≥1 Process
Data Flow	ID, Label, From, To	Connects components (respecting DFD rules)

Rule Enforcement: DFD rules validated in real-time. Invalid connections show error immediately.

Diagram: Auto-generated Mermaid graph from components and flows.

Node 7: ERD

Based on: Peter Chen model, Crow's Foot notation (Mermaid erDiagram)

Component	Fields
Entity	ID, Name (UPPERCASE), Description, Attributes list
Attribute	Name (snake_case), Data Type, Constraints (PK/FK/Unique/Nullable/Required/Index), Description
Relationship	EntityA, Cardinality (1:1/1:N/N:1/N:M), EntityB, Label (verb), Junction Table (auto for N:M)

N:M handling: Auto-suggest junction table. User confirms or renames. Junction entity auto-created.

SQL Paste: Alternative input tab. Paste CREATE TABLE → auto-parse into entities + attributes.

Task Engine: Entity → migration task. Relationship → setup task. N:M → pivot table task. Entity → seed data task.

Node 8: Sequence Diagram

Based on: UML 2.5 Sequence Diagram, combined fragments (alt/opt/loop)

Component	Fields
Sequence	ID, Name, Related UC, Related Flowchart (optional)
Participant	Name, Type (actor/component/service/database/external), Order
Message	Step#, From, To, Label, Type (request/response/self), Group (alt/opt/loop + label)

API Auto-extraction: Messages containing HTTP methods (GET/POST/PUT/DELETE) or /api/ paths are auto-extracted as API endpoint list.

Task Engine: HTTP messages → API endpoint tasks. Service participants → module tasks. Alt groups → error handling tasks.

Node 9: Task Board

Consumer node. Aggregates auto-generated + manual tasks. Supports 3 grouping modes (by User Story / by Layer / by Feature Flow). Full CRUD. Export to MD/CSV/JSON/Linear CSV.

Regeneration: Preserves manual tasks (is_manual=true). Deletes and re-generates auto tasks.

Node 10: Summary

Fully read-only, auto-calculated. Shows: project overview, completion status per node, all validation warnings, requirements coverage percentages, task summary, API endpoints, export buttons.

Done condition: ALL nodes Done AND validation warnings = 0. This is the ultimate gate before coding.

3. Cross-Validation Rules (18 Rules)

These rules run whenever any node is saved. Results shown as badges on nodes + in Validation Summary Panel.

ID	Source	Target	Warning
CV-01	Brief.targetUsers	User Stories.role	Target user has no stories
CV-02	Brief.targetUsers	Use Case.actor	Target user not in any use case
CV-03	Brief.scopeIn	Requirements.FR	Scope item has no requirements
CV-04	Requirements.FR	User Stories	FR has no user story
CV-05	User Stories	Use Cases	US has no use case
CV-06	Use Cases	Flowcharts	UC has no flowchart
CV-07	Use Cases	Sequence Diagrams	UC has no sequence diagram
CV-08	DFD.dataStores	ERD.entities	Data store has no matching entity
CV-09	DFD.extEntities	Brief.targetUsers	External entity not in target users
CV-10	DFD.processes	Use Cases	DFD process has no related UC
CV-11	ERD.entities	DFD.dataStores	Entity has no matching data store
CV-12	ERD.entities	US + UC	Entity not referenced in stories/cases
CV-13	ERD.entities	ERD.relationships	Entity is orphan (no relationships)
CV-14	Seq.participants(db)	ERD.entities	DB participant has no matching entity
CV-15	Seq.participants(ext)	DFD.extEntities	External participant not in DFD
CV-16	User Stories	Task Board	US has no generated tasks
CV-17	ERD.entities	Task Board	Entity has no database task
CV-18	UC include/extend	Use Cases	Referenced UC does not exist

▶ **Impact:** These 18 rules ensure that all nodes are synchronized. No more “ERD has 5 entities but flowchart only covers 2” situations.

4. Task Engine Extraction Rules

Source Node	Rule	Task Title Pattern	Auto Labels
User Story	Each US → 1 task	"Implement: [want]"	from FR category
US Acceptance	Each Given-When-Then → 1 task	"Test: Given [g], When [w], Then [t]"	testing
ERD Entity	Each entity → 1 task	"Create [Entity] migration + model"	database

ERD Entity 5+	Extra if 5+ attributes	"Add validations for [Entity]"	database
ERD Relationship	Each relationship → 1 task	"Setup [A] ↔ [B] relationship"	database
ERD N:M	Each junction → 1 task	"Create [Junction] pivot table"	database
ERD Entity	Each entity → seed task	"Create seed data for [Entity]"	database
Flowchart Process	Each process step → 1 task	"[step label]"	backend/frontend
Flowchart Decision	Each decision → 1 task	"Validate: [label]"	backend
Sequence HTTP	Each API message → 1 task	"Create [METHOD] [endpoint]"	backend
Sequence Service	Each service → 1 task	"Setup [service] module"	backend
Sequence Alt	Each alt group → 1 task	"Handle: [label] error case"	backend
DFD Process	Each process → 1 task	"Implement [name] logic"	backend
DFD External Sys	Each ext system → 1 task	"Integrate with [name]"	backend

Deduplication

Multiple nodes may generate similar tasks (e.g., Flowchart “Create account” and Sequence “POST /api/register”). Task Engine flags potential duplicates by title similarity. User decides to merge or keep separate.

5. Updated ERD (Archway Internal)

Archway’s own data model, updated for v3 with validation and structured content.

```
erDiagram
    PROJECTS ||--|{ NODES : contains
    PROJECTS ||--|{ EDGES : contains
    NODES ||--o| NODE_CONTENTS : has
    NODES ||--o{ TASKS : generates
    NODES ||--o{ ATTACHMENTS : has
    NODES ||--o{ VALIDATION_WARNINGS : produces

    NODE_CONTENTS {
        string id PK
        string node_id FK
        json structured_fields  "replaces guided_fields + free_text"
        string mermaid_auto     "auto-generated, read-only"
        string mermaid_manual   "user override, optional"
    }

    TASKS {
        string id PK
        string project_id FK
        string source_node_id FK
        string source_item_id
        string title
        string description
        string priority
        string labels JSON
```

```
        string status
        boolean is_manual
        string group_key
    }

    VALIDATION_WARNINGS {
        string id PK
        string project_id FK
        string source_node_id FK
        string target_node_type
        string severity
        string message
        string rule_id  "CV-01 through CV-18"
    }
```

Key change: NODE_CONTENTS.structured_fields is a JSON column that stores the typed, structured data for each node type. The shape of this JSON varies by node type but is always known and parseable.

New: VALIDATION_WARNINGS table stores cross-validation results. Recalculated on every node save.

New: NODE_CONTENTS.mermaid_auto stores the auto-generated Mermaid syntax (read-only, regenerated from structured_fields). mermaid_manual allows user override if they want custom Mermaid.

6. Validation UX Design

Node Badges in Pipeline

Each node in React Flow canvas shows status + validation:

State	Badge	Meaning
Empty, no warnings	○ Gray circle	Not started
In Progress, no warnings	◔ Yellow half-circle	Being worked on
Done, no warnings	✅ Green check	Complete and valid
Any status + warnings	⚠ Yellow triangle + count	Has validation issues
Recommended next	Subtle green glow	Soft guidance

Inside Editor Panel

Bottom of every editor panel shows node-specific warnings:

```
┌──────────────────────────────┐
│ Warnings (3):                │
│ ⚠ FR-005 has no user story   │
│ ⚠ FR-009 has no user story   │
│ ⚠ Scope item "Chat" has no FR │
└──────────────────────────────┘
```

Validation Summary Panel

Accessible via toolbar button. Shows ALL warnings across ALL nodes with direct links to the problematic node/field.

7. Revised Sprint Plan

The structured input overhaul changes the sprint plan. Sprints 1-2 (basic setup) from v2 are still valid. This updates Sprint 3 onwards.

Sprint	Focus	Key Deliverables
Sprint 3	Structured Node Editors	Replace all free-text with structured fields for all 10 nodes
Sprint 4	Auto-generated Diagrams	Mermaid auto-generation from structured data for UC, Flowchart, DFD, ERD, Sequence
Sprint 5	Validation System	Per-node validation + 18 cross-validation rules + validation summary panel
Sprint 6	Task Engine v2	Deterministic extraction from structured data + deduplication + Task Board UI
Sprint 7	Summary + Export	Auto-compiled summary with coverage % + all export formats
Sprint 8	Polish + Prepare Open Source	Bug fixes, README, LICENSE (MIT), CONTRIBUTING.md, demo project

▶ **v3 transforms Archway from a documentation tool into an intelligent documentation system.** Every input is structured. Every node is validated. Every gap is detected. Every task is traceable to its source. This is the bridge between thinking and building.